I.U.T. de Rodez
BUT Informatique – Semestre 3
Sujet proposé par : C. Servières

Année 2026 - 2027

# Projet Programmation Java

## Quiz test de connaissances

## 1. Contexte et objectif

### 1.1 Contexte

Le but de ce projet est de permettre aux utilisateurs de tester leurs connaissances grâce à des quiz. Le domaine des connaissances à tester n'est pas fixé ; il pourrait être l'histoire, la géographie, l'art, les sciences, le sport par exemple. De plus, le logiciel développé pourrait être utilisé par des étudiants pour réviser leurs cours, sous une forme ludique. Notamment dans le cadre du BUT Informatique, les étudiants de première année pourraient consolider leurs connaissances dans le langage Java et tous les étudiants progresser en orthographe, avec des questions adaptées à cet usage.

Le logiciel sera donc très général et paramétrable pour permettre des révisions sur des sujets divers. Il sera développé dans le langage Java.

### 1.2 Résumé des objectifs

Les questions présentes dans l'application pouvant être nombreuses, elles seront réparties dans des catégories. L'ajout des questions au sein de l'application pourra se faire de 3 manières différentes :

- Manuellement : l'utilisateur saisira dans l'application toutes les informations décrivant la question
- Par importation d'un fichier de questions présent sur l'ordinateur de l'utilisateur
- Par importation via un ordinateur distant. Dans ce cas, les données échangées seront chiffrées.

L'application devra mémoriser les derniers tests (c'est-à-dire les réponses données à un questionnaire) réalisés. Le but est de permettre à l'utilisateur de refaire un test antérieur.

Notons que dans cette version, l'application ne doit pas gérer plusieurs utilisateurs différents.

# 2. Exigences fonctionnelles

## 2.1 Dictionnaire

### Catégorie

Une catégorie est un regroupement (éventuellement vide) de questions. L'application possèdera une catégorie nommée « Général ». Cette catégorie ne pourra jamais être supprimée. L'utilisateur pourra en créer de nouvelles. On n'autorisera pas d'homonyme pour les noms de catégorie.

Exemple de noms : « Bases du langage Java», « Les villes de France », « Tennis », « Cinéma »

### Question

Toute question doit appartenir à une unique catégorie. Dans tous les cas, une question sera constituée :

- D'un niveau de difficulté : 1, 2, ou 3 pour les questions les plus difficiles
- D'un libellé
- D'une unique proposition de réponse juste
- D'au moins une proposition de réponse fausse et au plus 4
- Et éventuellement d'un texte de feedback qui sera affiché en tant que corrigé de la question

Par exemple, dans la catégorie « Bases du langage Java», on pourrait trouver une question de niveau 1 dont le libellé serait *Quel est le type de l'expression suivante '9' ?* et les propositions de réponse : *String* *int* *char*

### Banque de questions

Dans la suite du document, on nommera « banque de questions » l'ensemble des questions gérées par l'application et qui pourront être utilisées par le joueur pour tester ses connaissances.

### Questionnaire

Un questionnaire comporte :

- ✓ Un niveau de difficulté, éventuellement celui-ci pourra être « indifférent »
- ✓ Une catégorie de questions facultative
- ✓ Un nombre de questions souhaité : 5, 10, ou 20
- ✓ Un ensemble de questions correspondant aux 3 éléments précédents. Éventuellement, il se peut que le nombre de questions du questionnaire soit inférieur au nombre souhaité. Cette situation se produira si la banque de questions de l'application ne permet pas de constituer un questionnaire avec toutes les questions requises

### Tentative (ou test)

On appelle tentative un questionnaire auquel un utilisateur a répondu. Une tentative est donc constituée d'un questionnaire et des réponses (une réponse pouvant être vide) que l'utilisateur a faites aux questions.

## 2.2 Fonctionnalités

### 2.2.1 Gestion des catégories

Il sera possible de :
- Consulter l'ensemble des catégories gérées par l'application
- Consulter les questions appartenant à une catégorie
- Créer une nouvelle catégorie, à condition qu'il n'y ait pas homonyme
- Modifier le nom d'une catégorie, avec la même contrainte
- Supprimer une catégorie. Si celle-ci contient des questions, l'application en informera l'utilisateur qui pourra confirmer ou pas la suppression demandée. La suppression de la catégorie entraîne celle des questions qu'elle contient.

En revanche, il ne sera pas possible de modifier le nom de la catégorie « Général », ni de la supprimer.

### 2.2.2 Gestion des questions

Il sera possible de :
- Consulter l'ensemble des questions gérées par l'application et appartenant à une catégorie précise
- Consulter le détail d'une question
- Créer de nouvelles questions. L'interface devra être la plus ergonomique possible pour que l'utilisateur puisse sélectionner facilement la catégorie et le niveau de la question. Deux questions identiques ne peuvent pas être présentes dans l'application. Deux questions seront considérées comme identiques si elles ont la même catégorie, le même libellé et les mêmes propositions de réponse, donc le niveau n'intervient pas, de même que le feedback.
- Modifier la question. Toutes les sous-informations qui la composent sont modifiables. Il est donc possible également de déplacer la question dans une autre catégorie.
- Supprimer une question.

### 2.2.3 Importation des questions

La saisie des questions est une tâche fastidieuse. On souhaite proposer des facilités pour les intégrer à l'application. L'idée est la suivante. On pourra préparer au préalable une liste de questions dans un outil bureautique de type tableur (Excel par exemple). Les questions seront présentées dans un format bien précis. On trouvera une question par ligne et les colonnes préciserons dans l'ordre :

- ✓ La catégorie
- ✓ Le niveau
- ✓ Le libellé de la question

- ✓ La proposition juste
- ✓ Une proposition fausse
- ✓ Éventuellement sur les colonnes suivantes d'autres propositions fausses (au plus 4, au total)
- ✓ Éventuellement un texte de feedback qui sera affiché en même temps que le corrigé d'un questionnaire (voir plus loin)

Ce tableau ainsi constitué sera enregistré au format *csv* sur l'ordinateur sur lequel l'application est installée. L'application devra donc être capable d'intégrer le contenu d'un tel fichier dans la banque de questions. Bien sûr, il faudra envisager le cas où le format du fichier serait incorrect. Un message d'erreur précis informera de l'échec de l'importation du fichier, ou de certaines lignes, et expliquera la cause.

Si le fichier contient des catégories qui n'existent pas encore au sein de l'application, elles seront créées automatiquement. L'utilisateur sera informé de la création de ces nouvelles catégories. S'il existe déjà une question identique (voir définition dans la section « dictionnaire »), la question ne sera pas importée. L'utilisateur sera informé.

### 2.2.4 Préparation d'un questionnaire par un joueur

Avant de lancer un questionnaire, le joueur devra au préalable choisir les caractéristiques de celui-ci en précisant :

- ✓ Un niveau de difficulté, éventuellement celui-ci pourra être « indifférent »
- ✓ Une catégorie de questions. Le joueur pourra ne pas faire ce choix. Les questions seront alors choisies dans l'ensemble des catégories.
- ✓ Le nombre de questions auxquelles il souhaite répondre : 5, 10, ou 20.

Comme dit précédemment, il se peut que le nombre effectif de questions du questionnaire soit inférieur au nombre souhaité. Dans ce cas, un message l'informera de cette particularité, lui indiquera le nombre de questions que l'application est en mesure de lui soumettre, et lui demandera s'il souhaite poursuivre.

Bien sûr, cette dernière demande ne doit être faite que s'il y a au moins une question répondant aux critères.

### 2.2.5 Lancement d'un questionnaire par un joueur

Une fois les paramètres du questionnaire spécifiés, l'application affichera les questions choisies au hasard parmi celles de la banque de questions et répondant aux souhaits de l'utilisateur en termes de difficulté et de catégorie. Notons que les propositions de réponses elles aussi seront affichées dans un ordre aléatoire.

Pour toute question, le joueur est autorisé à ne pas répondre. Il peut dans ce cas passer à la question suivante. Mais bien sûr la non-réponse à la question sera comptabilisée parmi les réponses incorrectes. Une fois le questionnaire validé, l'application affichera le résultat :

- Le nombre de réponses justes
- Le pourcentage de réponses justes
- Une phrase adaptée au score obtenu : « Félicitations ! Toutes vos réponses sont justes ! », « Bravo ! plus de la moitié de vos réponses sont justes ! » …

Elle proposera également au joueur de visualiser les réponses correctes. C'est lors de cette visualisation que le feedback sera affiché, s'il a été renseigné.

### 2.2.6 Communication entre deux ordinateurs

Comme dit précédemment, on souhaite faciliter la tâche des utilisateurs qui souhaitent partager tout ou partie des questions (avec leur catégorie).

L'application Quiz doit pouvoir, suivant le choix de son utilisateur, soit envoyer des questions, soit recevoir des questions. Pour ce faire, l'application Quiz devra au préalable établir une connexion avec un pair déjà lancé.

Les données envoyées seront chiffrées en utilisant un algorithme de chiffrement symétrique, Vigenère ou Hill, une seule de ces méthodes étant effectivement à implémenter. Au préalable, un échange de Diffie-Hellman permettra aux deux ordinateurs distants de partager une donnée secrète. La clé de chiffrement symétrique utilisée sera dérivée de ce secret partagé.

L'utilisateur doit avoir la possibilité d'envoyer soit toutes les questions de la banque de questions, soit uniquement celles qu'il aura sélectionnées. Il pourra sélectionner au choix une ou plusieurs catégories, ou encore individuellement certaines questions.

Lors de l'importation, des vérifications devront être faites : format de la question, éventuellement création d'une nouvelle catégorie, vérification qu'une question identique n'existe pas déjà. Pour ces vérifications, le comportement de l'application sera similaire à celui adopté lors de l'importation d'un fichier *csv* local (voir dans les paragraphes précédents).

Précisions

On suppose que :
- l'échange de données n'est possible qu'entre 2 pairs à la fois
- les 2 pairs sont préalablement intégrés à un même réseau (en particulier, du point de vue IP)
- l'application doit disposer d'une rubrique, d'une étape, … permettant d'afficher et si besoin modifier ses paramètres réseau (adresse IP et numéro de port). Ceux-ci devront être préalablement et automatiquement préétablis

Contraintes techniques

- utiliser des sockets Java en mode connecté avec les classes *Socket* et *ServerSocket*
- utiliser du multithreading pour ne pas bloquer l'application pendant le transfert des données
- une connexion par échange de fichier, arrêtée sans intervention de l'utilisateur à la fin de l'échange

### 2.2.7 Gestion de l'historique

L'application devra mémoriser les 10 dernières tentatives réalisées. Cette valeur 10 est la valeur par défaut et doit être modifiable via les paramètres de l'application.

Le joueur pourra ainsi refaire une tentative antérieure. L'idée est bien sûr qu'il améliore ainsi son niveau de connaissances. A la fin de la nouvelle tentative, un message complémentaire lui indiquera sa

progression par rapport à la tentative précédente (éventuellement sa régression), en lui rappelant également son score précédent.

Remarque importante : l'application devra gérer au mieux le cas particulier où l'une des questions aurait été supprimée ou modifiée depuis son utilisation dans la tentative. Aucune gestion particulière n'est imposée dans ce cahier des charges. En accord avec votre client, vous choisirez une manière adéquate pour prendre en compte ce cas particulier.

A la demande du joueur, l'application devra afficher l'historique des tests passés. Pour chacun d'eux, les informations importantes à mentionner sont la date et l'heure du test, la catégorie, le niveau et le pourcentage de bonnes réponses.

Le joueur doit avoir la possibilité de supprimer un test de l'historique.

### 2.2.8 Fonctionnalité facultative : Mode perfectionnement avancé

Éventuellement (au choix de l'équipe de développement), l'application permettra à l'utilisateur d'entrer dans le mode de jeu « Perfectionnement avancé ». Dans ce cas, il ne choisira pas une catégorie, mais seulement un nombre de questions parmi 5, 10, 20 ou 30, et éventuellement un niveau de difficulté. L'application lui soumettra alors des questions choisies de manière aléatoire parmi celles auxquelles il a donné une mauvaise réponse, et présentes dans l'historique des tentatives.

Il n'est pas imposé de conserver un historique des tests passés dans le mode « Perfectionnement avancé ». Toutefois, vous pourrez ajouter cette fonctionnalité à l'application si vous le souhaitez.

### 2.2.9 Autres fonctionnalités

L'utilisateur aura la possibilité de visualiser une notice d'utilisation de l'application. Celle-ci expliquera notamment le format du fichier Excel permettant l'importation de questions.

Via les paramètres, l'utilisateur pourra renseigner son nom. Celui-ci permettra de personnaliser les messages informant des résultats, notamment.

L'application sera livrée avec une banque de questions initiales que vous aurez conçue et d'autres qui vous seront fournies (certaines porteront sur le langage Java, d'autres sur l'orthographe).

## 3.Exigences non fonctionnelles

La persistance des données (donc de la banque de questions, des catégories, de l'historique …) sera assurée grâce à la sérialisation. Si nécessaire, un support de cours vous sera donné. Il n'est donc pas demandé de mettre en place une base de données.

L'interface graphique devra être aussi ergonomique que possible. On gèrera au mieux les erreurs ou anomalies qui pourraient se produire.

# 4.Livrables

---

## ATTENTION

**Des livrables destinés à Mme Fraison, M. Palosse et M. Rous sont identiques pour les 2 projets, Java et Web, et sont décrits dans un document annexe**

---

## 4.1 Livrable « Développement » - Compétence 1 – Réaliser – AC1 et AC3 (M. Kam et Mme Servières)

Les livrables sont les suivants :

- tout le code source de l'application, y compris les programmes de test, via un dépôt Github et aussi une archive compressée à déposer sur Moodle
- un exécutable de l'application avec toutes les dépendances associées (l'application doit se lancer via un simple clic) à déposer sur Moodle
- un dossier technique au format *pdf* à déposer sur Moodle

Le **dossier technique** au format *pdf* comportera :

- une page de garde
- un sommaire
- une introduction : résumé de l'application réalisée (une demi-page au moins et jusqu'à une page)
- un diagramme de cas d'utilisation UML
- la conception de l'application avec un diagramme de classes UML
- une présentation des tests unitaires réalisés. Pour chaque test unitaire, il faut indiquer :
  - le nom de la classe testée et son paquetage
  - le nom de la classe de test unitaire et son paquetage
  - des explications précisant en quoi les jeux de tests et les algorithmes de tests que vous avez codés sont pertinents
- les tests d'intégration effectués ou bien les jeux d'essai utilisés pour tester l'application finale. Cette partie du dossier est destinée à juger de votre capacité à présenter des tests d'intégration. Il n'est donc pas demandé de présenter les tests de toute l'application. Vous choisirez 2 fonctionnalités (ou User Stories) et vous présenterez les tests d'intégration effectués pour ces 2 fonctionnalités. Toutefois, vous ne choisirez pas une fonctionnalité trop simple, comme « Afficher l'aide pour l'utilisateur ». Le mieux sera de demander une validation à Mme Servières avant de rédiger cette partie. Pour les 2 fonctionnalités retenues, vous devez expliquer les interactions de l'utilisateur, dans différents cas (conditions normales et anormales), et montrer le comportement de l'application.

  Vous pouvez utiliser des copies d'écran accompagnées d'explications. Si certains cas testés ont conduit à un résultat identique, il n'est pas utile de mettre une copie d'écran pour chacun d'eux.

Vous indiquerez toutefois quels sont les cas qui ont conduit à un résultat similaire à la copie d'écran. Le but est de démontrer la bonne qualité de votre couverture de tests.

Cette partie du dossier technique doit être à la fois complète et synthétique.

- un bilan faisant état des difficultés rencontrées et des solutions apportées
- une conclusion pour **chacun** des étudiants du groupe et éventuellement une conclusion globale au groupe

La qualité de la rédaction (clarté, précision, mise en forme, orthographe) sera prise en compte dans l'évaluation.

## 4.2 Livrable « Conception » - Compétence 1 – Réaliser – AC3 (M. Pomeret)

Un dossier technique décrivant la conception de l'application sera remis. Il comportera :

- une page de garde
- éventuellement, une brève introduction
- un ou plusieurs diagrammes de classes UML illustrant l'application complète. Vous veillerez à ce que ces diagrammes soient lisibles et pertinents.
- des explications sur vos choix de conception. Au besoin, les choix de conception peuvent être illustrés chacun par un ou plusieurs diagrammes détaillés (lisibles et pertinents).

## 4.3 Livrable « chiffrement » - Compétence 3 – Administrer – AC1 (M. Wehrlé)

Un document au format *pdf*, correctement présenté et rédigé, qui doit contenir :

- Toutes les précisions nécessaires sur la méthode de chiffrement symétrique effectivement mise en œuvre
- Un diagramme de séquence présentant les différentes étapes nécessaires au chiffrement/déchiffrement des données (côté ordinateur qui exporte les données et côté ordinateur distant)
- La manière avec laquelle la clé de chiffrement symétrique est dérivée de l'entier partagé suite à l'échange de Diffie-Hellman ; vous illustrerez cela avec au moins un exemple
- Le code source (seulement celui directement lié au chiffrement), bien commenté

## 4.4 Livrable « présentation en anglais » - Compétence 6 – Collaborer – AC4 (Mme Alberge)

Un support powerpoint que vous présenterez à l'oral, correctement présenté et rédigé. La présentation en groupe durera 10 minutes et sera suivie d'un temps de questions/réponses individuelles.

Vous pouvez choisir de présenter un seul des deux projets ou bien les deux.

**Option A** : focus sur le projet web ou le projet Java

**Option B** : présentation synthétique des deux projets

- expliquez en quoi consistait la mission (objectifs, contraintes imposées, tout élément d'information qui vous semble utile pour que le sujet du projet soit le plus clair possible)
- expliquez comment vous avez procédé
- indiquez le temps de travail par semaine (celui-ci peut varier d'un étudiant à l'autre)
- précisez comment vous vous êtes réparti les tâches
- évoquez les réunions et les points d'étape que vous avez organisés, avec le groupe de travail et avec le client. Indiquez en quoi ces mises au point vous ont permis d'avancer de manière plus efficace et plus constructive

Vous avez droit à des notes personnelles mais il s'agit d'un oral : votre présentation doit être suffisamment anticipée, préparée et maîtrisée pour ne pas avoir à lire. La dimension orale sera fortement évaluée :

- fluidité de l'expression
- maîtrise du contenu
- contact visuel avec le jury, prise en compte des réactions de l'auditoire
- efforts d'élocution et d'articulation
- débit de parole
- intonation adaptée au contenu